

Project 2 - Fourier Transform

Joshnare A. Ong

Group Members: Raffy Colobong & Nathan Ocampo

ENGG 151.01 - C

October 3, 2026

The project was implemented in C++ using Code::Blocks 20.03. The program was developed to import a discrete-time signal from a file and calculate its Discrete Fourier Transform (DFT) over a user-specified frequency range. The program calculates the real and imaginary components of the DFT as well as its magnitude and phase

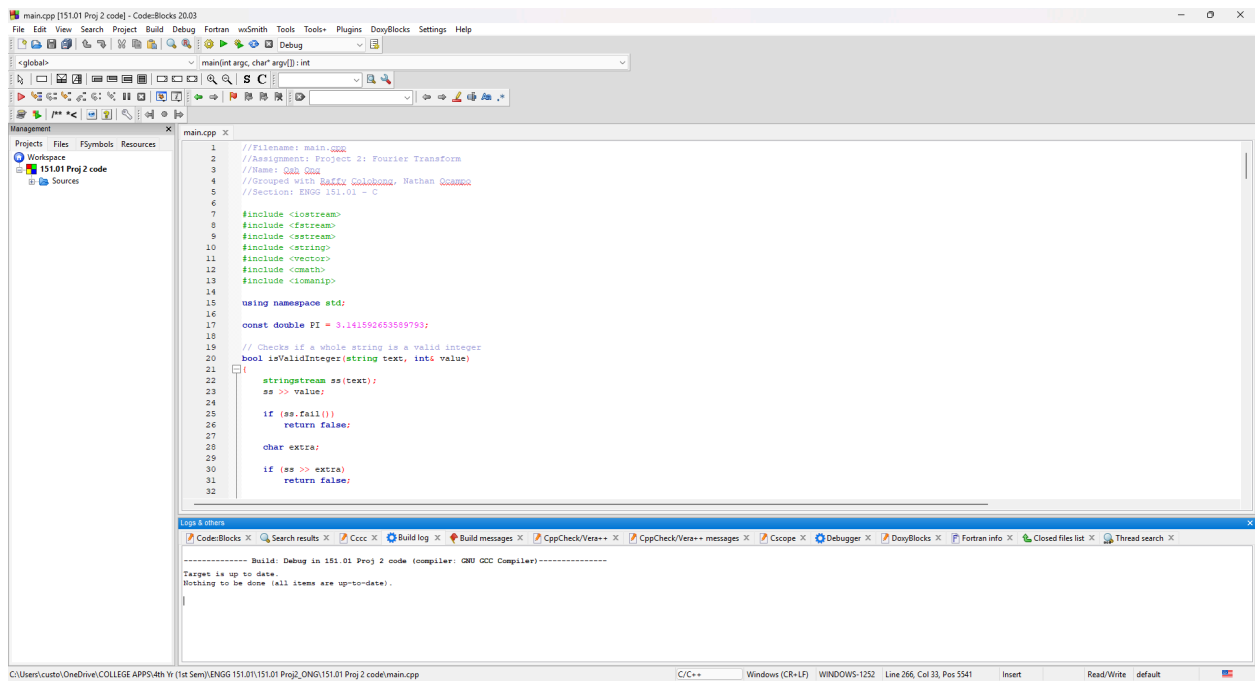


Figure 1. Successful Code

The program was tested using the provided Basic Test with the argument: x.signal 32 4 8 8

[illegible]

The calculated results were also written to `dfilog.txt`. The program separates the results into two tables: frequency with real and imaginary values, followed by frequency magnitude and phase. A custom log filename may also be provided, and existing log files are appended instead of overwritten.

File	Edit	View		
Frequency	Real	Imaginary	Magnitude	Phase
4	-16.0164	-1.27133	16.0668	-175.462
4.5	-2.09174	23.9087	24	95
5	14.5237	1.37905	14.589	5.42409
5.5	-4.25822e-10	1.3819e-09	1.44602e-09	107.126
6	4.45829	0.479847	4.48404	6.1431
6.5	1.97938e-09	-1.09724e-09	2.26316e-09	-29.0012
7	2.49867	0.291875	2.51566	6.66266
7.5	1.84839e-09	-9.68995e-10	2.0799e-09	-27.7675
8	1.69007	0.206077	1.70258	6.95199
frequency	real part	imaginary part		
4	-16.0164	-1.27133		
4.5	-2.09174	23.9087		
5	14.5237	1.37905		
5.5	-4.25822e-10	1.3819e-09		
6	4.45829	0.479847		
6.5	1.97938e-09	-1.09724e-09		
7	2.49867	0.291875		
7.5	1.84839e-09	-9.68995e-10		
8	1.69007	0.206077		
frequency	magnitude	phase		
4	16.0668	-175.462		
4.5	24	95		
5	14.589	5.42409		
5.5	1.44602e-09	107.126		
6	4.48404	6.1431		
6.5	2.26316e-09	-29.0012		
7	2.51566	6.66266		
7.5	2.0799e-09	-27.7675		
8	1.70258	6.95199		
frequency	real part	imaginary part		
4	-16.0164	-1.27133		
4.5	-2.09174	23.9087		
5	14.5237	1.37905		
5.5	-4.25822e-10	1.3819e-09		
6	4.45829	0.479847		
6.5	1.97938e-09	-1.09724e-09		
7	2.49867	0.291875		
7.5	1.84839e-09	-9.68995e-10		
8	1.69007	0.206077		
frequency	magnitude	phase		
4	16.0668	-175.462		
4.5	24	95		
5	14.589	5.42409		
5.5	1.44602e-09	107.126		
6	4.48404	6.1431		
6.5	2.26316e-09	-29.0012		
7	2.51566	6.66266		
7.5	2.0799e-09	-27.7675		
8	1.70258	6.95199		
frequency	real part	imaginary part		
4	-16.0164	-1.27133		
4.4	-9.27989	21.9573		
4.8	15.0563	13.2083		
5.2	7.14473	-4.19475		
5.6	0.409176	1.90501		
6	4.45829	0.479847		
6.4	0.43882	-0.968423		
6.8	1.64926	1.51531		
7.2	1.62468	-0.910242		
7.6	0.117027	0.594606		
8	1.69007	0.206077		
frequency	magnitude	phase		
4	16.0668	-175.462		
4.4	23.8378	112.911		
4.8	20.0207	41.2592		
5.2	8.18511	-30.4177		
5.6	1.94845	77.8776		
6	4.48404	6.1431		
6.4	1.06321	-65.6234		
6.8	2.23969	42.5762		
7.2	1.86229	-29.2601		
7.6	0.606013	78.8657		
8	1.70258	6.95199		
frequency	real part	imaginary part		
4	-16.0164	-1.27133		
4.4	-9.27989	21.9573		
4.8	15.0563	13.2083		
5.2	7.14473	-4.19475		
5.6	0.409176	1.90501		
6	4.45829	0.479847		
6.4	0.43882	-0.968423		
6.8	1.64926	1.51531		
7.2	1.62468	-0.910242		
7.6	0.117027	0.594606		

Figure 3. DFT Log File

The program was also tested for input handling. Invalid command-line arguments were successfully detected. When an invalid value was encountered within a signal file, signal

extraction stopped at that point while retaining the valid samples that appeared before the invalid entry.

Table 1. Code Locator

Code Item	File Name	Line Numbers
Correlation function	main.cpp	131 - 184
Declaration of double * passed to the correlation function	main.cpp	240 264 - 267
Separate processing of first line of signal file	main.cpp	64 - 102
Input validation of integers in processing signal files	main.cpp	20 - 34
Input validation of floating-point numbers in processing signal files	main.cpp	37 - 51 85 95 115

In conclusion, the project successfully implemented a C++ program for computing the DFT of an imported signal. The required Basic Test produced the expected results, while logfile output, command-line processing, dynamic memory allocation, and input validation were also implemented.

Project Evaluation - Normalized Crosscorrelation		
Item	Points	Rubrics

early work	8/8	(all or nothing) 8: at most 10% of the code in the final implementation differs from that in early work submission
implementation and testing environment report	4/4	4 - all instructions followed correctly
Basic Test	50/50	50 - correctly computes the DFT of basic test data
	6/6	6 - screenshot submitted showing the correctly computed DFT of basic test data as it appears on the console output
generality of DFT	6/6	6 - the DFT is computed correctly and efficiently at all times
DFT function	6/6	6 - the DFT is computed correctly and efficiently at all times, by a function as specified
processing of signal files	4/4	4 - The file is opened only once and the first line of the signal file is processed correctly and separately from code processing the rest of the signal file
input validation	4/4	4 - integers and floating point numbers are fully validated when signal files are processed
command line	4/4	4 - application processes command line arguments correctly at all times
application feedback	4/4	4 - application provides appropriate feedback at all times, as specified
self-evaluation	4/4	4 - self-evaluation accurate (or evaluating this item leads to an error)
total	100/100	